

Réécriture de code récursif à l'aide de boucles

en langage Caml Light

Présentation de **Lorem Ipsum (50792)**

travail réalisé avec **Dolor sit amet (15980)**

Fonctions récursives et boucles

- ▶ On a souvent besoin d'itérer des instructions sur des données, c'est-à-dire de parcourir une certaine séquence de données en les traitant une par une

Fonctions récursives et boucles

- ▶ On a souvent besoin d'itérer des instructions sur des données, c'est-à-dire de parcourir une certaine séquence de données en les traitant une par une
- ▶ Deux méthodes existent : boucles et fonctions récursives

Boucles

Avantages :

Boucles

Avantages :

- ▶ Plus facile à compiler ou à interpréter ;
- ▶ Moins de problèmes mémoires.

Boucles

Avantages :

- ▶ Plus facile à compiler ou à interpréter ;
- ▶ Moins de problèmes mémoires.

Exemple :

```
1  let somme_liste (l: int list) =  
2    let resultat = ref 0 in  
3    let l' = ref l in  
4    while !l' <> [] do  
5      resultat := !resultat + List.hd !l';  
6      l' := List.tl !l'  
7  done;  
8  !resultat
```

Fonctions récursives

- Avantage : parfois préférable en terme de lisibilité de code.

Fonctions récursives

- ▶ Avantage : parfois préférable en terme de lisibilité de code.
- ▶ Problème : peut entraîner des dépassements de pile.

Fonctions récursives

- ▶ Avantage : parfois préférable en terme de lisibilité de code.
- ▶ Problème : peut entraîner des dépassements de pile.

Exemple :

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Passage des fonctions récursives aux boucles

Passage des fonctions récursives aux boucles

- ▶ Étape classique de l'optimisation lors de la compilation de langages fonctionnels

Passage des fonctions récursives aux boucles

- ▶ Étape classique de l'optimisation lors de la compilation de langages fonctionnels
- ▶ Comment ça marche en pratique ?

Passage des fonctions récursives aux boucles

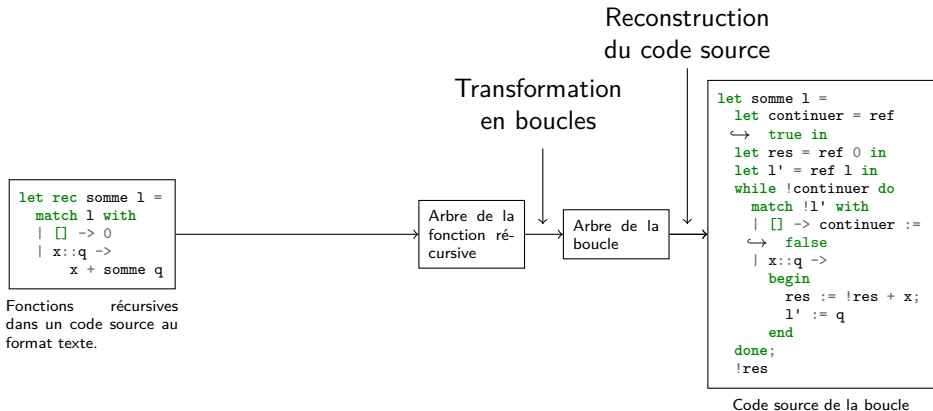
```
let rec somme l =
  match l with
  | [] -> 0
  | x::q ->
    x + somme q
```

Fonctions récursives
dans un code source au
format texte.

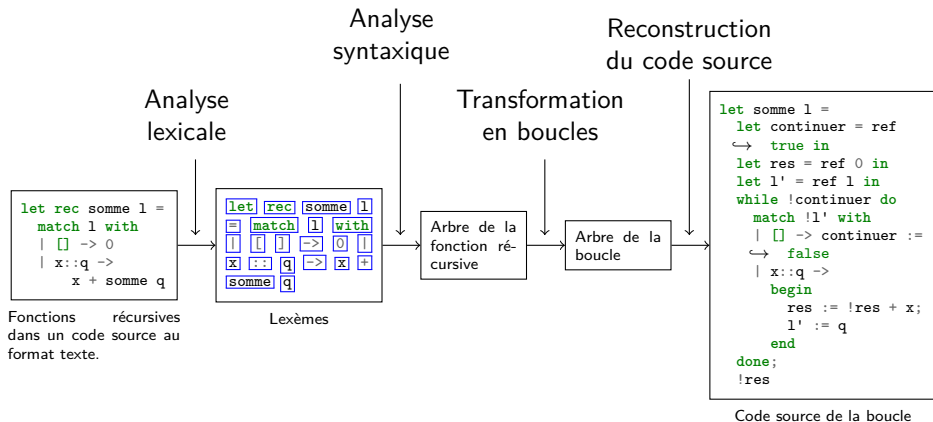
```
let somme l =
  let continuer = ref
  ↪ true in
  let res = ref 0 in
  let l' = ref l in
  while !continuer do
    match !l' with
    | [] -> continuer :=
  ↪ false
    | x::q ->
      begin
        res := !res + x;
        l' := q
      end
  done;
  !res
```

Code source de la boucle

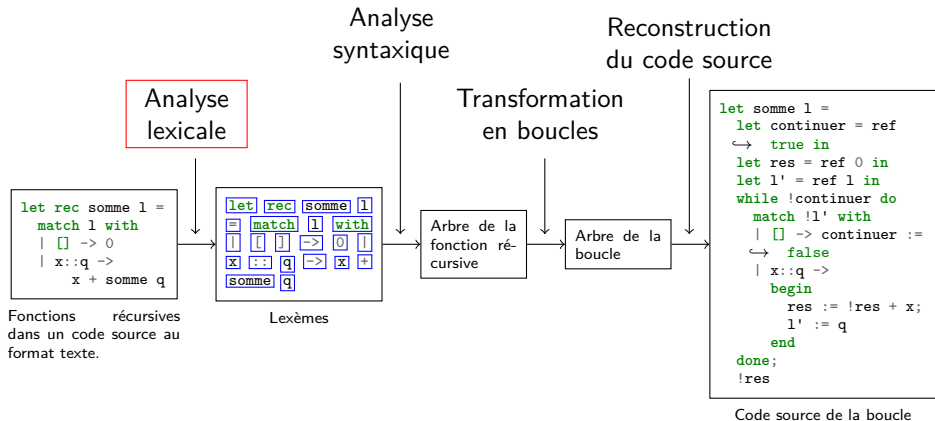
Passage des fonctions récursives aux boucles



Passage des fonctions récursives aux boucles



Plan



Analyse lexicale

But : construire une liste de lexèmes à partir du code source.

Analyse lexicale

But : construire une liste de lexèmes à partir du code source.

Définition (lexème)

Couple (type, valeur).

*Les types de lexèmes sont également appelés **symboles terminaux**.*

Analyse lexicale

But : construire une liste de lexèmes à partir du code source.

Définition (lexème)

Couple (type, valeur).

*Les types de lexèmes sont également appelés **symboles terminaux**.*

Exemples : (int, 12), (string, "Hello, world"), (equals, =).

Analyse lexicale

Exemple :

```
let f x = x
```

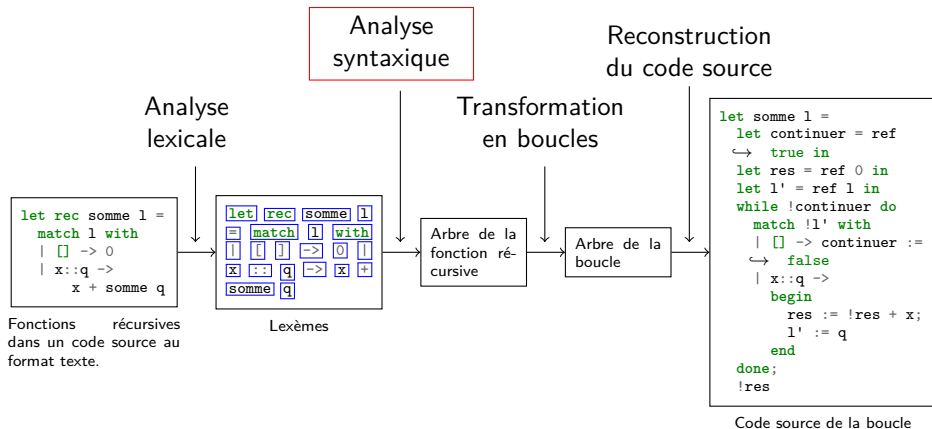
Analyse lexicale

Exemple :

```
let f x = x
```

```
(let, let),  
(identifiant, f),  
(identifiant, x),  
(equals, =),  
(identifiant, x)
```

Plan



Analyse syntaxique

But : construire un arbre de syntaxe à partir de la liste de lexèmes.

Analyse syntaxique

But : construire un arbre de syntaxe à partir de la liste de lexèmes.

Exemple :

```
let f x = x
```

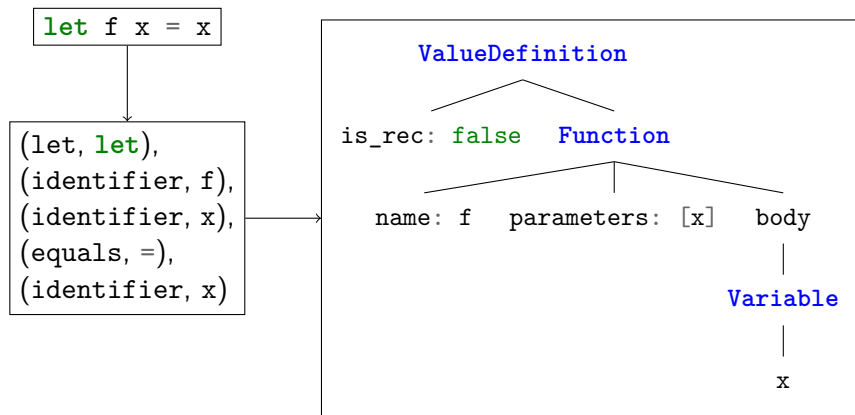


```
(let, let),  
(identifiant, f),  
(identifiant, x),  
(equals, =),  
(identifiant, x)
```


Analyse syntaxique

But : construire un arbre de syntaxe à partir de la liste de lexèmes.

Exemple : Note : faut-il montrer un arbre de syntaxe construit par LR(0) plutôt qu'un AST ?



Grammaires

Pour décrire les « règles de branchement » dans nos arbres, on introduit la notion de grammaire.

Grammaires

Pour décrire les « règles de branchement » dans nos arbres, on introduit la notion de grammaire.

Définition (grammaire)

- Ensemble de règles (dites « règles de dérivation ») de la forme $\boxed{\text{Symbole} \rightarrow \dots}$;

Grammaires

Pour décrire les « règles de branchement » dans nos arbres, on introduit la notion de grammaire.

Définition (grammaire)

- ▶ Ensemble de règles (dites « règles de dérivation ») de la forme $\boxed{\text{Symbole} \rightarrow \dots}$;
- ▶ Les symboles à gauche sont appelés **symboles non terminaux** ;

Grammaires

Pour décrire les « règles de branchement » dans nos arbres, on introduit la notion de grammaire.

Définition (grammaire)

- ▶ Ensemble de règles (dites « règles de dérivation ») de la forme $\boxed{\text{Symbole} \rightarrow \dots}$;
- ▶ Les symboles à gauche sont appelés **symboles non terminaux** ;
- ▶ À droite de la flèche : liste de **symboles terminaux et non terminaux**.

Exemple

PHRASE → VARIABLE égal EXPR

VARIABLE → identifiant

EXPR → littéral_d'entier

EXPR → parenthese_g EXPR OPÉRATEUR EXPR parenthese_d

OPÉRATEUR → plus

OPÉRATEUR → fois

Exemple

PHRASE → VARIABLE égal EXPR

VARIABLE → identifiant

EXPR → littéral_d'entier

EXPR → parenthese_g EXPR OPÉRATEUR EXPR parenthese_d

OPÉRATEUR → plus

OPÉRATEUR → fois

Exemple de « code source » :

$$x = (3 \times (1 + 2))$$

Exemple

PHRASE → VARIABLE égal EXPR

VARIABLE → identifiant

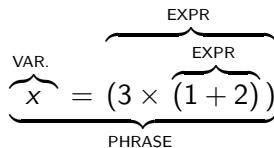
EXPR → littéral_d'entier

EXPR → parenthese_g EXPR OPÉRATEUR EXPR parenthese_d

OPÉRATEUR → plus

OPÉRATEUR → fois

Exemple de « code source » :



Exemple

PHRASE → VARIABLE égal EXPR

VARIABLE → identifiant

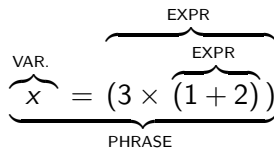
EXPR → littéral_d'entier

EXPR → parenthese_g EXPR OPÉRATEUR EXPR parenthese_d

OPÉRATEUR → plus

OPÉRATEUR → fois

Exemple de « code source » :



Problème : comment passer d'une grammaire et d'un code source à un arbre de syntaxe ?

Automates

Automates

L'algorithme LR(0) repose sur l'utilisation **d'automates**.

Automates

L'algorithme LR(0) repose sur l'utilisation **d'automates**.

Définition (automate)

*Un **automate LR(0)** est composé d'états et de transitions.*

*Les **états LR(0)** sont constitués de règles de grammaires comportant une position.*

*Les **transitions** entre les états peuvent être des symboles terminaux (types de lexèmes) ou des symboles non terminaux.*

Automates

L'algorithme LR(0) repose sur l'utilisation **d'automates**.

Définition (automate)

*Un **automate LR(0)** est composé d'états et de transitions.*

*Les **états LR(0)** sont constitués de règles de grammaires comportant une position.*

*Les **transitions** entre les états peuvent être des symboles terminaux (types de lexèmes) ou des symboles non terminaux.*

L'automate LR(0) correspondant à une grammaire peut être construit à partir de cette grammaire.

Automates

PHRASE → VARIABLE égal Expr

VARIABLE → identifiant

Expr → littéral_d'entier

Expr → parenthese_g Expr OPÉRATEUR Expr parenthese_d

OPÉRATEUR → plus

OPÉRATEUR → fois

Automates

$\underline{P} \rightarrow V \text{ égal } E$

$V \rightarrow \text{id}$

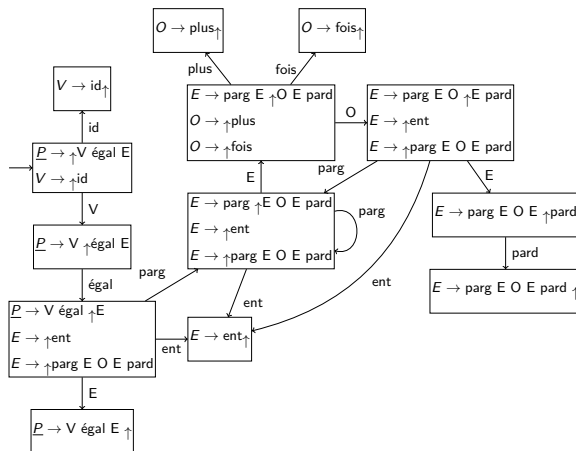
$E \rightarrow \text{ent}$

$E \rightarrow \text{parg } E \text{ O } E \text{ pard}$

$O \rightarrow \text{plus}$

$O \rightarrow \text{fois}$

Algorithme LR(0) sur un exemple



$$\underline{P} \rightarrow V \text{   gal } E$$

$$V \rightarrow id$$

$$E \rightarrow ent$$

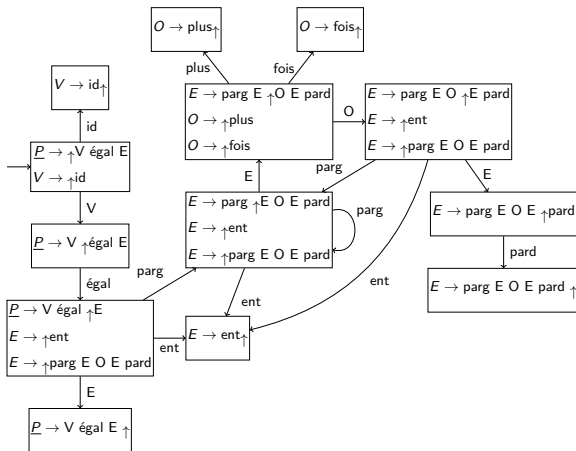
$$E \rightarrow parg \ E \ O \ E \ pard$$

$$O \rightarrow plus$$

$$O \rightarrow fois$$

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$



$\underline{P} \rightarrow V \text{ égal } E$

$V \rightarrow id$

$E \rightarrow \text{ent}$

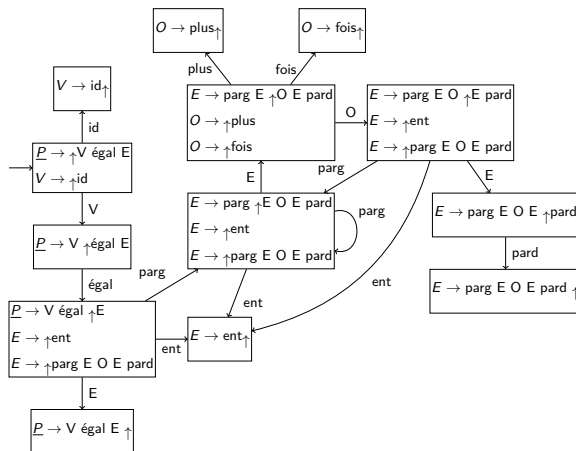
$E \rightarrow \text{parg } E \text{ O } E \text{ pard}$

$O \rightarrow \text{plus}$

$O \rightarrow \text{fois}$

Algorithme LR(0) sur un exemple

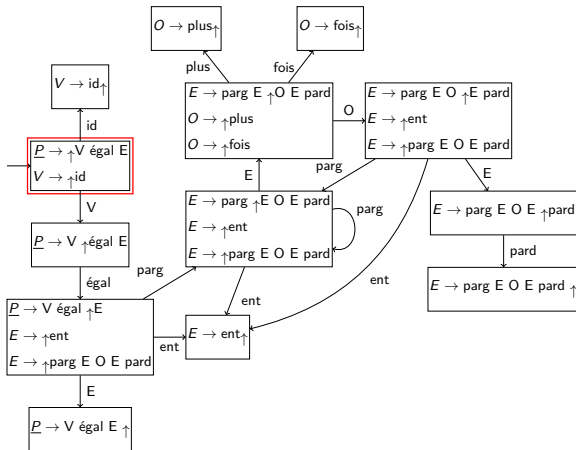
Exemple : $x = (3 \times (1 + 2))$



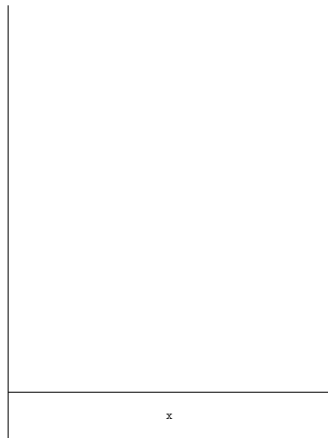
Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

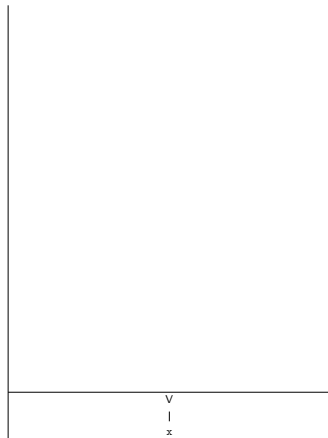
$\uparrow x = (3 \times (1 + 2))$



Exemple : $x = (3 \times (1 + 2))$ $x_{\uparrow} = (3 \times (1 + 2))$

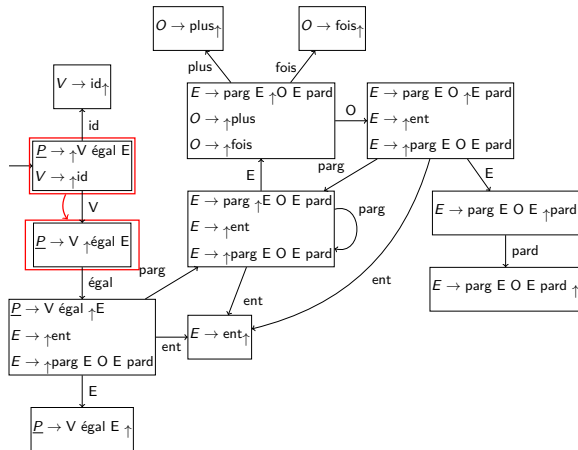


Exemple : $x = (3 \times (1 + 2))$ $\uparrow V = (3 \times (1 + 2))$



Exemple : $x = (3 \times (1 + 2))$

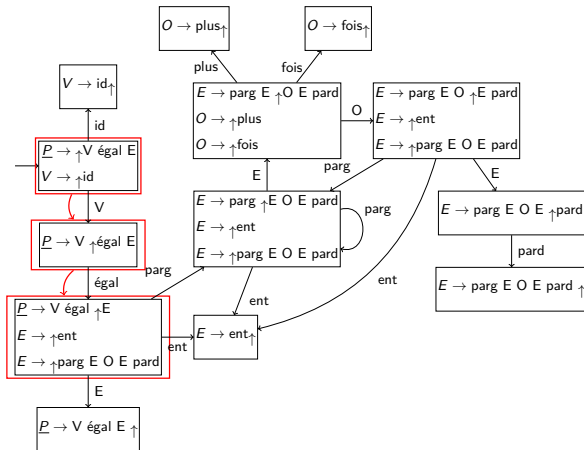
$$V_{\uparrow} = (3 \times (1 + 2))$$


$$\frac{V}{I} = X$$

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = \uparrow(3 \times (1 + 2))$

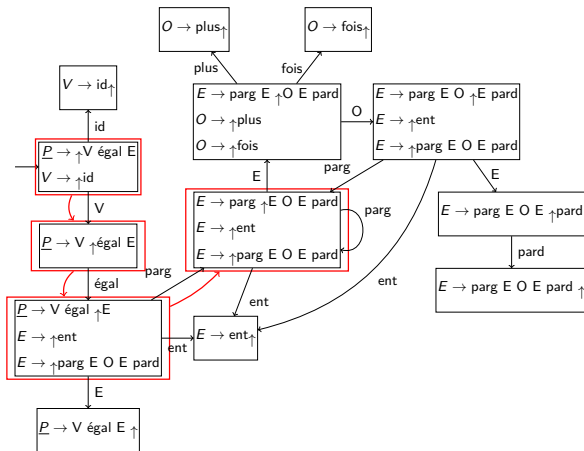


=
V
x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (\uparrow 3 \times (1 + 2))$

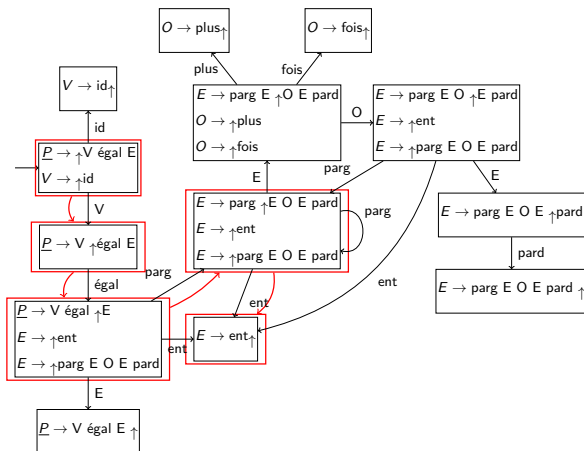


(
=
V
x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

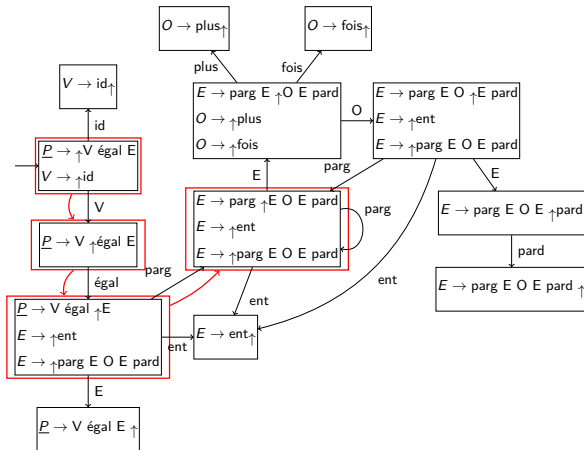
$V = (3_{\uparrow} \times (1 + 2))$



3
(
=
V
x

Exemple : $x = (3 \times (1 + 2))$

$$V = (\uparrow E \times (1 + 2))$$

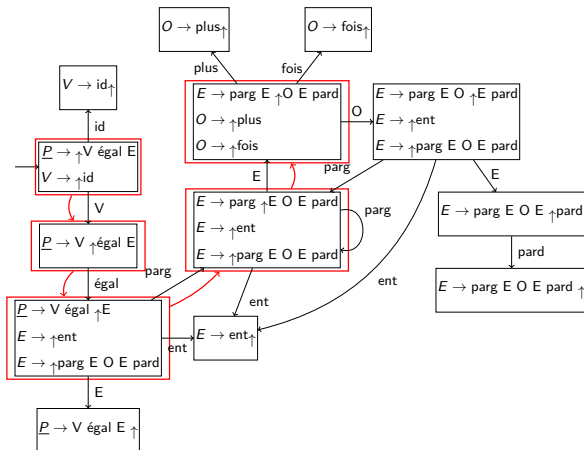


	E
	3
	(
	=
	V
	x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

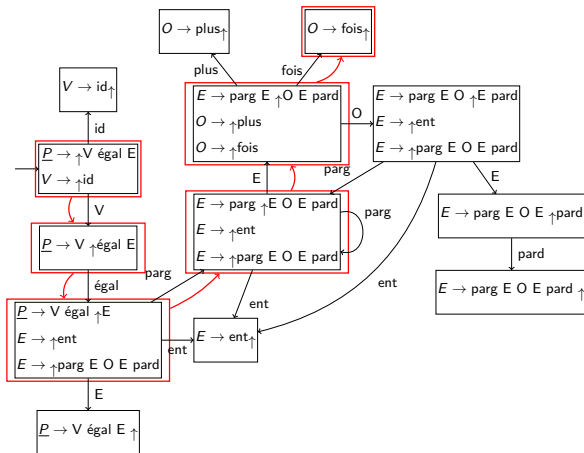
$V = (E_{\uparrow} \times (1 + 2))$



E
3
(
=
V
x

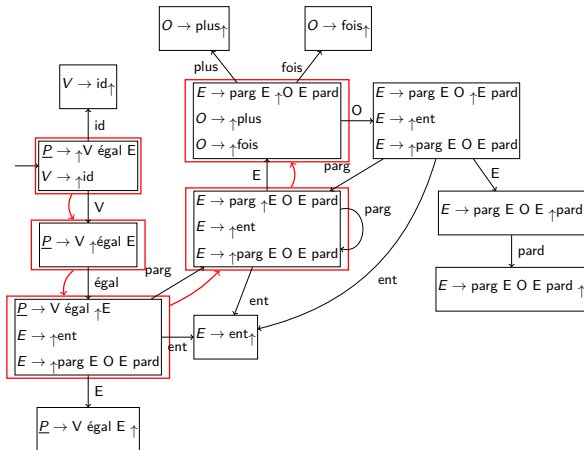
Exemple : $x = (3 \times (1 + 2))$

$$V = (E \times \uparrow(1 + 2))$$



x
E 3
(
=
V x

Exemple : $x = (3 \times (1 + 2))$ $V = (E_{\uparrow} O(1 + 2))$

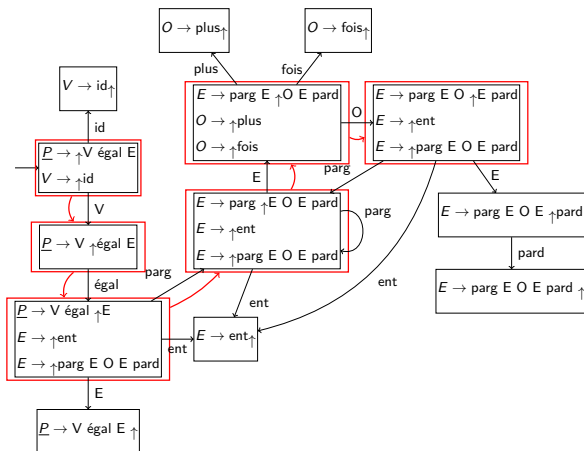


	O
	x
	E
	3
	(
	=
	V
	x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO \uparrow (1 + 2))$

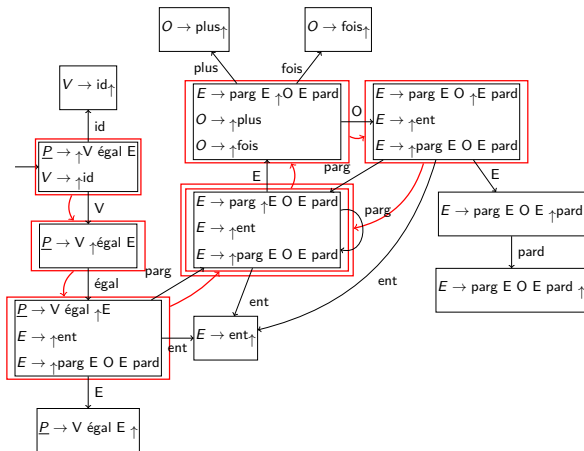


	O
	x
E	
	3
	(
	=
V	
	x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO(\uparrow 1 + 2))$

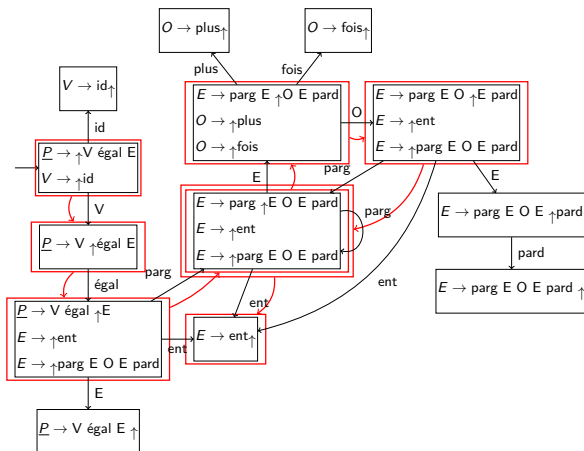


(
O
x
E
3
(
=
V
x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO(1\uparrow + 2))$

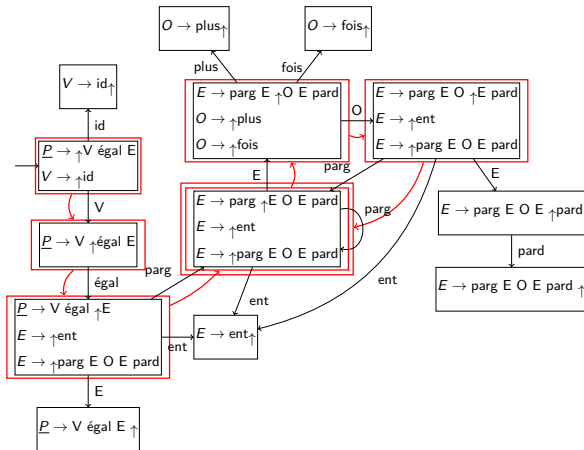


1
(
O
x
E
3
(
=
V
x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO(\uparrow E + 2))$

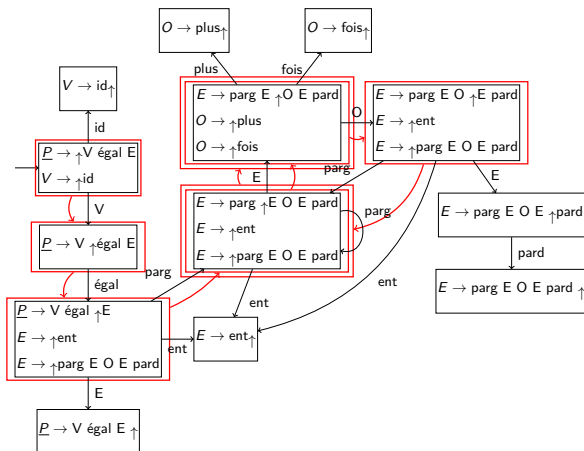


E
1
(
O
x
E
3
(
=
V
x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO(E_{\uparrow} + 2))$

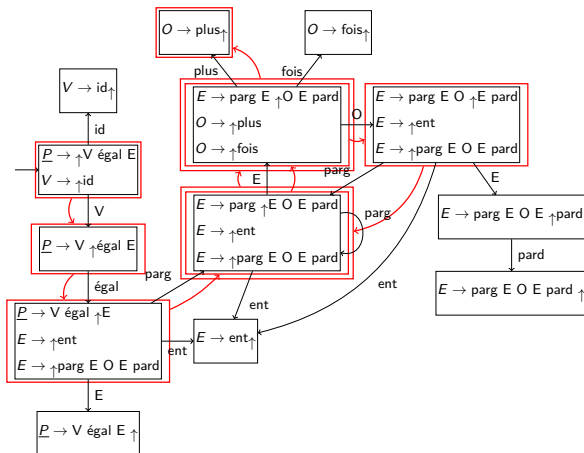


E
1
(
O
x
E
3
(
=
V
x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO(E + \uparrow 2))$

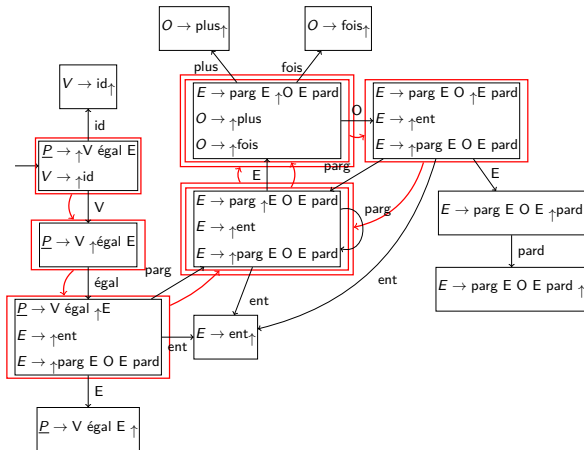


+
E 1
(
O x
E 3
(
=
V x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO(E \uparrow O2))$

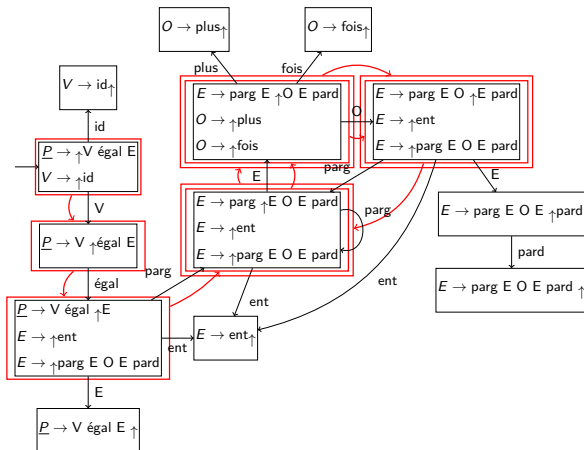


	O
	+
E	
	1
(	
O	
	x
E	
	3
(	
=	
V	
	x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO(EO \uparrow 2))$

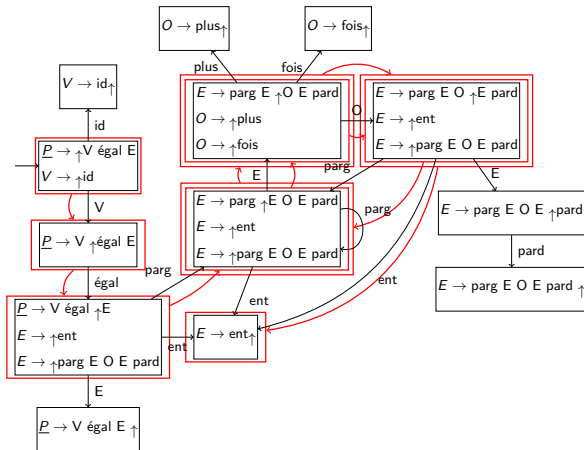


	O
	+
	E
	1
	(
	O
	x
	E
	3
	(
	=
	V
	x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO(EO2\uparrow))$

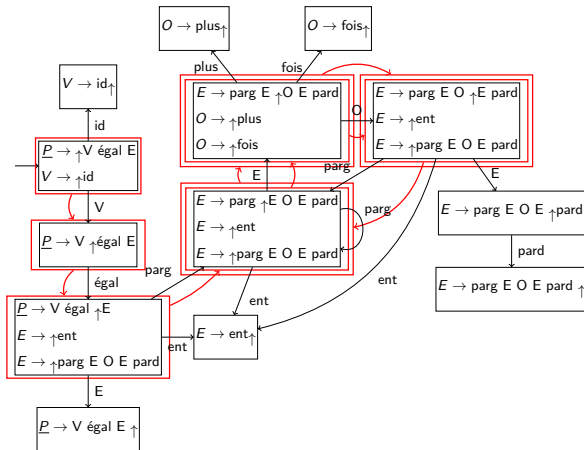


2
O
+
E
1
(
O
x
E
3
(
=
V
x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO(EO \uparrow E))$

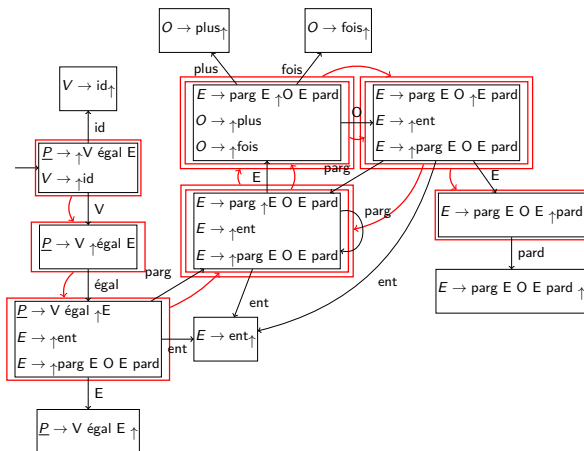


	E
	2
	O
	+
	E
	1
	(
	O
	x
	E
	3
	(
	=
	V
	x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EO(EOE\uparrow))$

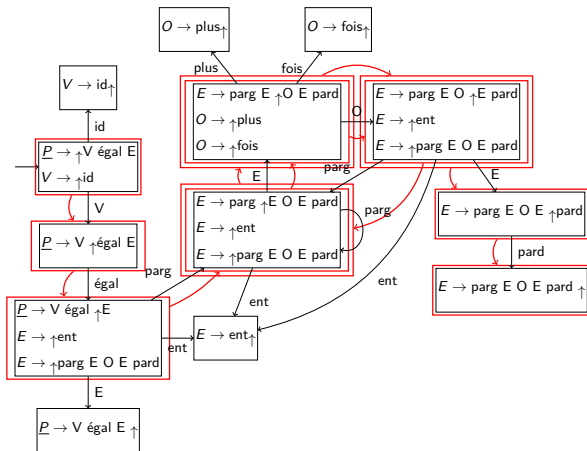


	E
	2
	O
	+
	E
	1
	(
	O
	x
	E
	3
	(
	=
	V
	x

Algorithme LR(0) sur un exemple

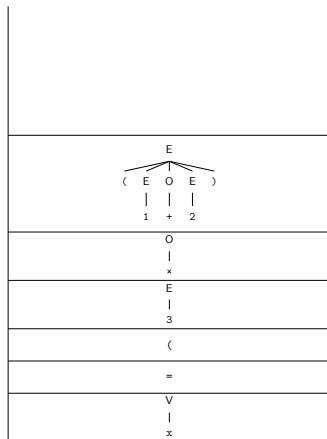
Exemple : $x = (3 \times (1 + 2))$

$V = (EO(EOE)\uparrow)$



	)
E	
2	
O	
+	
E	
1	
(	
O	
	x
E	
3	
(	
=	
V	
x	

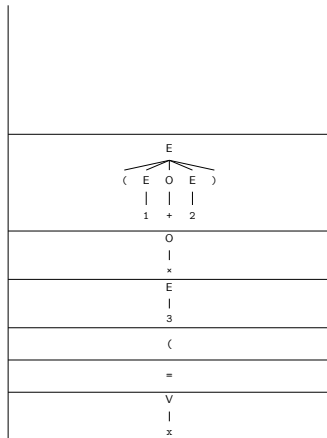
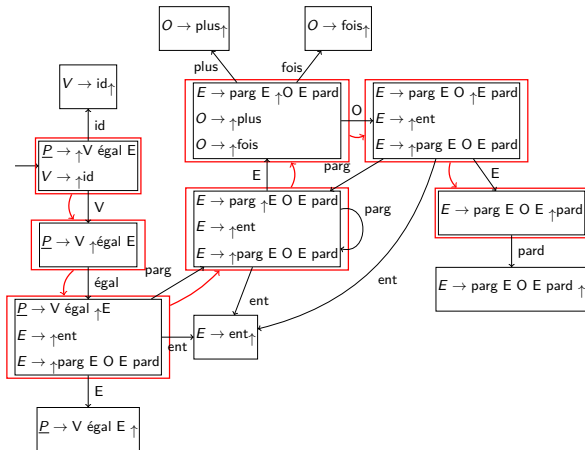
Exemple : $x = (3 \times (1 + 2))$ $V = (EO \uparrow E)$



Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

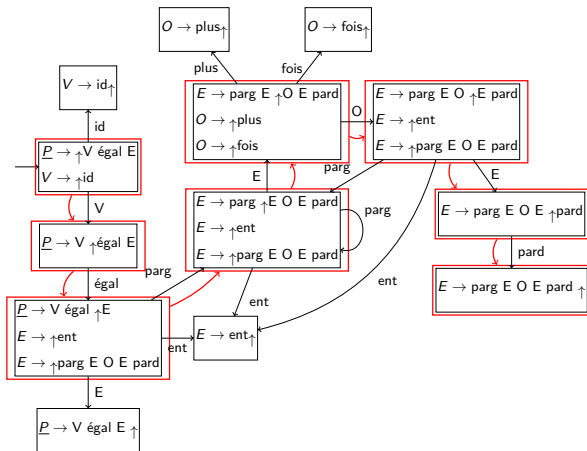
$V = (EOE \uparrow)$



Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (EOE)_{\uparrow}$

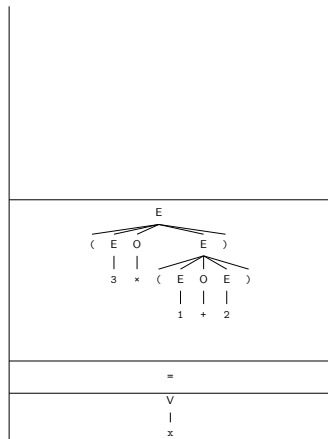
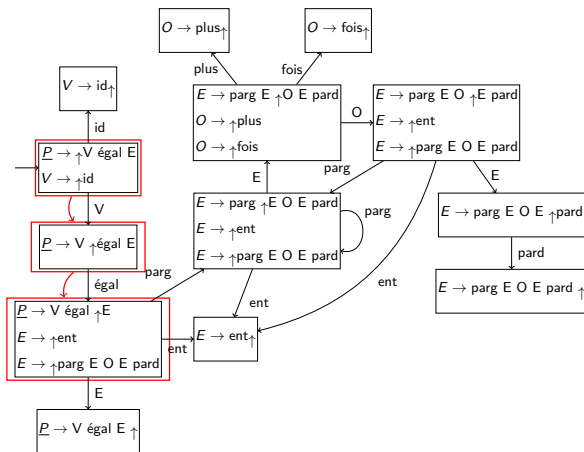


)
E (E O E) 1 + 2
O
x
E
3
(
=
V
x

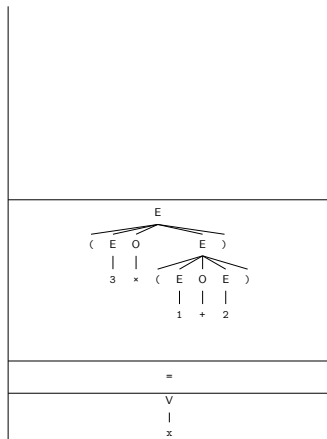
Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = \uparrow E$

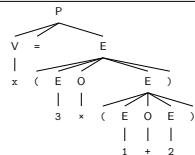
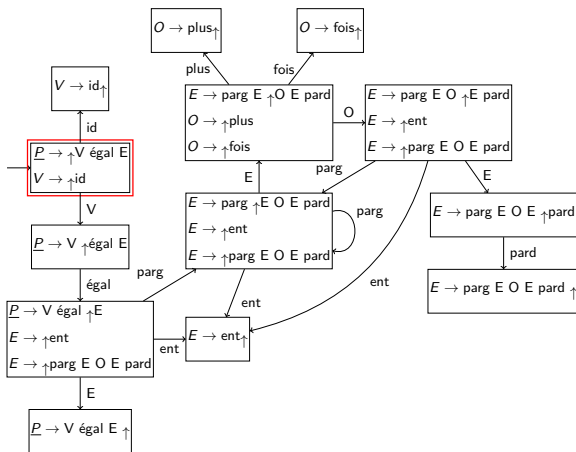


Exemple : $x = (3 \times (1 + 2))$ $V = E_{\uparrow}$



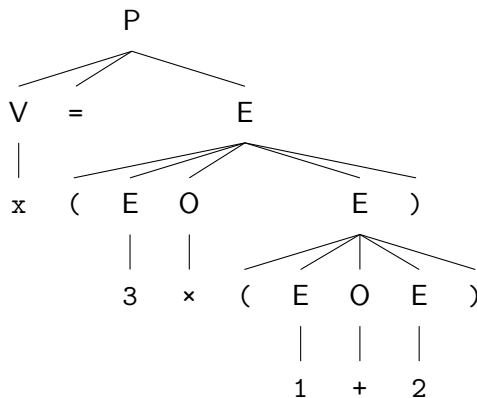
Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$ $\uparrow \underline{P}$

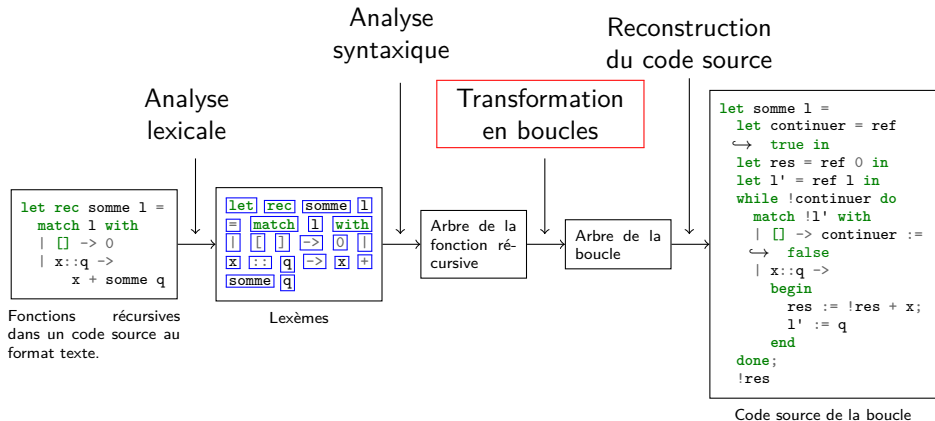


Algorithme LR(0) sur un exemple

$$x = (3 \times (1 + 2))$$

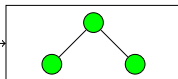


Plan



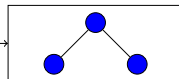
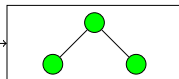
Mise sous forme de boucles

```
let rec somme l a =  
  match l with  
  | [] -> a  
  | x::q ->  
    somme q (a+x)
```



Mise sous forme de boucles

```
let rec somme l a =
  match l with
  | [] -> a
  | x::q ->
    somme q (a+x)
```



```
let somme l a =
  let c = ref true in
  let res = ref None in
  let l' = ref l in
  let a' = ref a in
  while !c do
    match !l' with
    | [] ->
      begin
        c := false;
        res := Some a'
      end
    | x::q ->
      begin
        l' := q;
        a' := !a'+x
      end
  end
  done;
  Option.get !res
```

Remplacement des appels récursifs

```
1  let rec somme l a =  
2  
3    match l with  
4    | [] -> a  
5    | x::q ->  
6      somme q (a+x)
```

Remplacement des appels récursifs

```
1 let rec somme l a =  
2     ←  
3     match l with  
4     | [] -> a  
5     | x::q ->  
6         somme q (a+x)
```

```
1 let c = ref true in  
2 let res = ref None in  
3 let l' = ref l in  
4 let a' = ref a in
```

Remplacement des appels récursifs

```
1 let rec somme l a =  
2   let c = ref true in  
3   let res = ref None in  
4   let l' = l in  
5   let a' = a in  
6   match l with  
7   | [] -> a  
8   | x::q ->  
9       somme q (a+x)
```

Remplacement des appels récursifs

```
1 let rec somme l a =  
2   let c = ref true in  
3   let res = ref None in  
4   let l' = l in  
5   let a' = a in  
6   match l with  
7   | [] -> a  
8   | x::q ->  
9     somme q (a+x)
```

Remplacement des appels récursifs

```
1 let rec somme l a =  
2   let c = ref true in  
3   let res = ref None in  
4   let l' = l in  
5   let a' = a in  
6   match l with  
7   | [] -> a  
8   | x::q ->  
9     somme q (a+x)
```

```
1 c := false;  
2 res := Some a'  
  
1 l' := q;  
2 a' := !a'+x
```


Remplacement des appels récursifs

```
1 let rec somme l a =  
2   let c = ref true in  
3   let res = ref None in  
4   let l' = l in  
5   let a' = a in  
6   match l with  
7   | [] -> a  
8   | x::q ->  
9     somme q (a+x)
```

```
1 c := false;  
2 res := Some a'  
  
1 l' := q;  
2 a' := !a'+x
```

Remplacement des appels récursifs

```
1  let somme l a =  
2    let c = ref true in  
3    let res = ref None in  
4    let l' = ref l in  
5    let a' = ref a in  
6    match !l' with  
7    | [] ->  
8      begin  
9        c := false;  
10       res := Some a'  
11      end  
12    | x::q ->  
13      begin  
14        l' := q;  
15        a' := !a'+x  
16      end
```

Remplacement des appels récursifs

```
1  let somme l a =  
2    let c = ref true in  
3    let res = ref None in  
4    let l' = ref l in  
5    let a' = ref a in  
6    while !c do  
7      match !l' with  
8      | [] ->  
9        begin  
10         c := false;  
11         res := Some a'  
12       end  
13      | x::q ->  
14        begin  
15         l' := q;  
16         a' := !a'+x  
17        end  
18    done;  
19    Option.get !res
```

Mise sous forme de boucles

```
1 | let rec somme l =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q ->  
5 |     x + somme q
```

```
1 | let rec somme l a =  
2 |   match l with  
3 |   | [] -> a  
4 |   | x::q ->  
5 |     somme q (a+x)
```

Problème

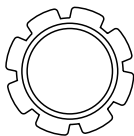
```
1  let rec somme l =  
2    match l with  
3    | [] -> 0  
4    | x::q ->  
5      x + somme q
```

Problème

```
1  let rec somme l =  
2    match l with  
3    | [] -> 0  
4    | x::q ->  
5      x + l' := q
```

Mise sous forme récursive terminale

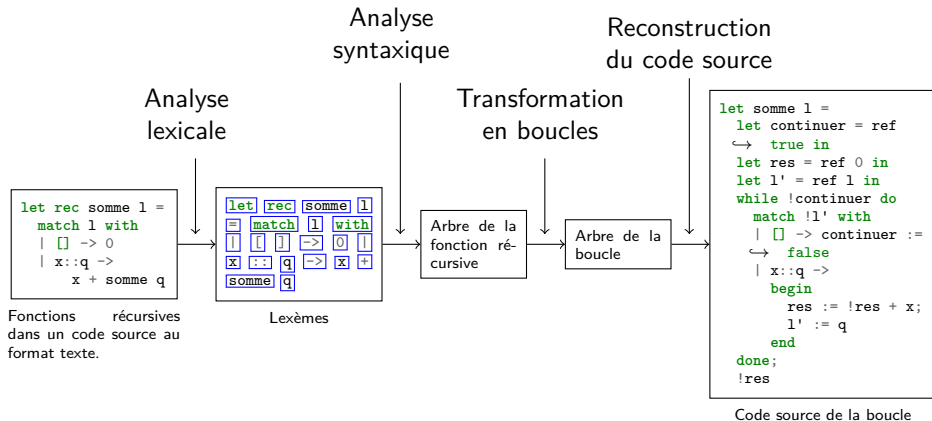
```
1  let rec somme l =  
2    match l with  
3    | [] -> 0  
4    | x::q ->  
5      x + somme q
```



Algorithme de
passage sous forme
récursive terminale

```
1  let rec somme l a =  
2    match l with  
3    | [] -> a  
4    | x::q ->  
5      somme q (a+x)
```

Plan



Conclusion

Résultats sur la somme d'une liste à 50×10^6 éléments :

Conclusion

Résultats sur la somme d'une liste à 50×10^6 éléments :

- ▶ Dépassement de pile avec la méthode récursive naïve
- ▶ Exécution en 5 secondes avec le code généré (utilisant une boucle)